# DESIGN AND ANALYSIS OF ALGORITHM

CAPSTONE PROJECT

NAME: NISHITHA PENAGALURU

ROLL NO: CH.SC.U4CSE24031

## Problem Statement Name:

105 - Pack up!

## Abstract:

Efficiently packing objects into limited space is an important optimization problem encountered in logistics, transportation, and storage management. This project focuses on solving a three-dimensional suitcase packing problem where a set of items with varying dimensions must be arranged inside a suitcase with fixed dimensions. The objective is to maximize the number of items packed while ensuring that items do not overlap and remain within the suitcase boundaries.

To solve this problem, a Backtracking algorithm is used. The algorithm explores possible placements of items by checking all valid positions and orientations while ensuring that spatial constraints are satisfied. Items can also be rotated to increase packing possibilities. The algorithm records the best arrangement that packs the maximum number of items.

This approach guarantees correctness for smaller input sizes and demonstrates important algorithmic concepts

such as recursion, search space exploration, and pruning techniques.

## Problem Statement:

Given a suitcase with fixed three-dimensional dimensions (width, breadth, and height) and a list of items with their respective dimensions, determine the optimal placement of items inside the suitcase to maximize the number of items packed.

The algorithm must ensure the following constraints:

- Items must not overlap with each other.
- All items must be completely inside the suitcase.
- Items may be rotated to fit better inside the suitcase.
- The algorithm must determine the spatial coordinates for each item placed.

The goal is to efficiently explore possible arrangements and find the configuration that fits the maximum number of items.

## Input:

The algorithm requires the following inputs:

1. Suitcase Dimensions
    a. Width (W)
    b. Breadth (B)
    c. Height (H)
2. Number of Items
    a. N
3. Dimensions of Each Item
    a. Width (Wi)
    b. Breadth (Bi)
    c. Height (Hi)

## Output:

The algorithm produces the following outputs:

1. Maximum number of items packed
2. Coordinates of each packed item

Each item's placement is represented using two coordinates:

- Bottom-front-left corner (x1, y1, z1)
- Top-back-right corner (x2, y2, z2)

## Limitations:

Although the algorithm guarantees optimal results for small inputs, it has certain limitations:

- The algorithm uses backtracking, which may explore many possible configurations.
- The worst-case time complexity grows exponentially as the number of items increases.
- For large numbers of items, the execution time may become very high.
- The algorithm does not guarantee the visually most compact arrangement, only the maximum item count.

## Objectives:

The main objectives of this project are:

- To design an algorithm that efficiently packs items into a suitcase.
- To ensure that items do not overlap and stay within boundaries.
- To maximize the number of items packed.
- To explore different item orientations through rotation.

- To demonstrate the use of backtracking algorithms in solving spatial optimization problems.

## Proposed Algorithm:

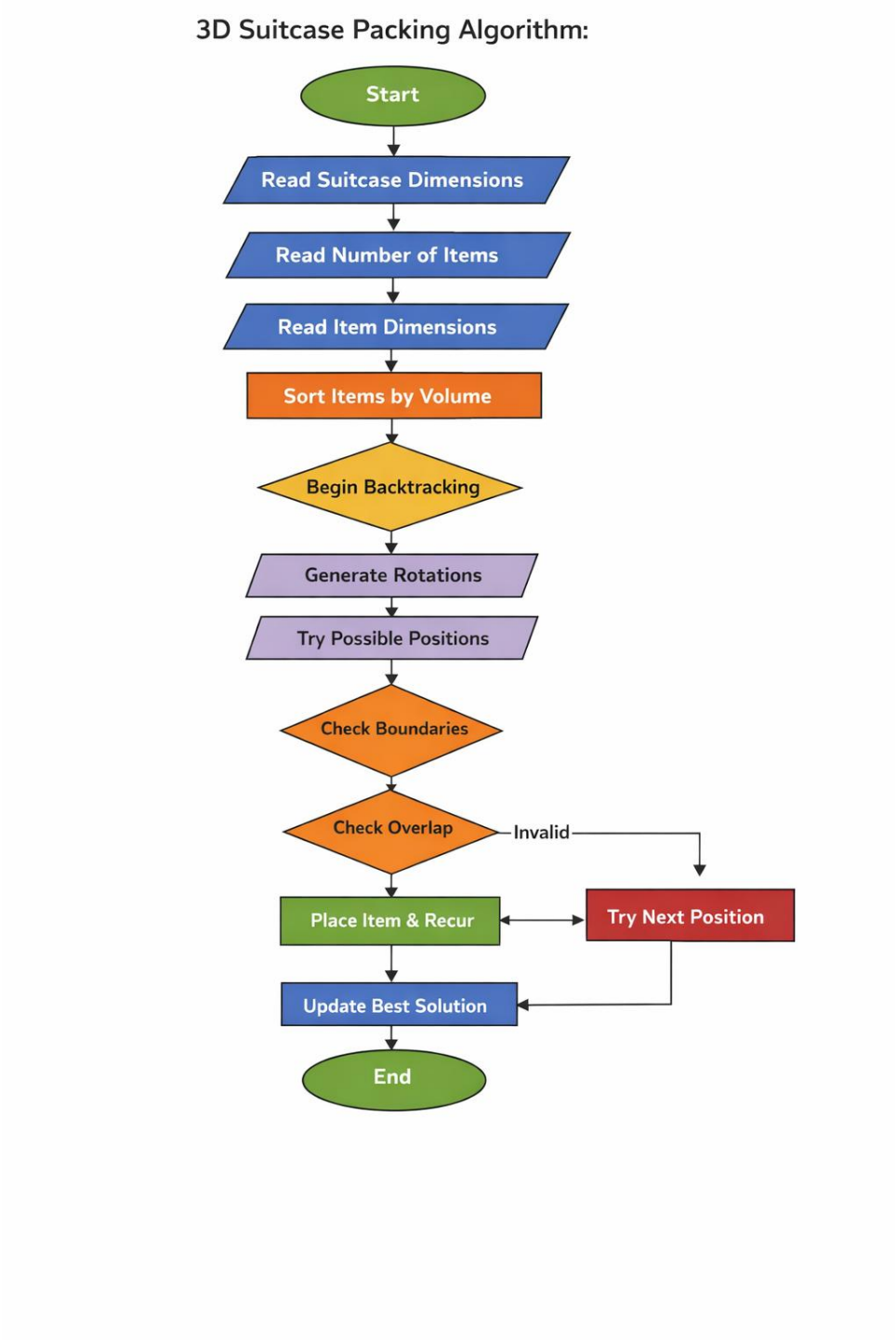
The proposed solution uses a Backtracking algorithm to explore possible placements of items.

Steps of the algorithm:

1. Sort items based on their volume in descending order.
2. Select the first item and generate its possible rotations.
3. Try placing the item at every valid position inside the suitcase.
4. Check if the placement causes overlap with already placed items.
5. If the placement is valid, place the item and continue with the next item.
6. If the placement is invalid, try the next possible position.
7. Use pruning to avoid exploring branches that cannot improve the current best solution.
8. Continue until all possibilities are explored.

The best configuration with the maximum number of items is stored as the final result.

# Flowchart:


3D Suitcase Packing Algorithm:
Start
Read Suitcase Dimensions
Read Number of Items
Read Item Dimensions
Sort Items by Volume
Begin Backtracking
Generate Rotations
Try Possible Positions
Check Boundaries
Check Overlap
Invalid
Place Item & Recur
Try Next Position
Update Best Solution
End

## Pseudocode:

```
function packItems(itemIndex, packedCount):

    update best solution if packedCount > maxPacked
    if itemIndex >= totalItems:

    return

    if packedCount + remainingItems <= maxPacked:

    return

    for each rotation of current item:

        for each position (x,y,z) in suitcase:

            if placement inside boundary:

                if no overlap with placed item
                    place item

                    packItems(itemIndex + 1, packedCount + 1)


    packItems(itemIndex + 1, packedCount)
```

## Implementation:

The program uses:

- Structures to store item dimensions and placements
- Recursion to implement backtracking
- Collision detection to prevent overlapping
- Sorting to prioritize larger items
- Pruning to reduce unnecessary computations

The implementation ensures that all constraints are respected while exploring possible packing arrangements.

# Code:

```
#include <stdio.h>

#define MAX_ITEMS 20

typedef struct { int w, b, h; int volume; } Item;

typedef struct { int x1, y1, z1; int x2, y2, z2; } Placement;

Item items[MAX_ITEMS];

Placement currentPlacement[MAX_ITEMS]; Placement
bestPlacement[MAX_ITEMS];

int suitcaseW, suitcaseB, suitcaseH; int totalItems;

int maxPacked = 0;

// Sort items by volume (descending) void sortItems() {

for (int i = 0; i < totalItems - 1; i++) {
    for (int j = i + 1; j < totalItems; j++) {

        if (items[i].volume < items[j].volume)
{

            Item temp = items[i];
            items[i] = items[j];
            items[j] = temp;
        }
```

```
    }
}


}
```

// Overlap check int isOverlap(Placement a, Placement b) {

```
if (a.x1 >= b.x2 || a.x2 <= b.x1)
    return 0;

if (a.y1 >= b.y2 || a.y2 <= b.y1)
    return 0;

if (a.z1 >= b.z2 || a.z2 <= b.z1)
    return 0;

return 1;


}
```

// Collision check int checkCollision(int placedCount, Placement newBox) {

```
for (int i = 0; i < placedCount; i++) {

    if (isOverlap(currentPlacement[i], newBox))
        return 1;
}
```

```
return 0;


}
```

// Backtracking void packItems(int itemIndex, int packedCount) {

```
// Update best result
if (packedCount > maxPacked) {

  maxPacked = packedCount;

    for (int i = 0; i < packedCount; i++)
bestPlacement[i] = currentPlacement[i];
}

// Stop condition
if (itemIndex >= totalItems)
    return;

// PRUNING
if (packedCount + (totalItems - itemIndex) <=
maxPacked)
    return;

int rotations[6][3] = {
    {items[itemIndex].w, items[itemIndex].b,
```

```
items[itemIndex].h},
    {items[itemIndex].w, items[itemIndex].h,
items[itemIndex].b},
    {items[itemIndex].b, items[itemIndex].w,
items[itemIndex].h},
    {items[itemIndex].b, items[itemIndex].h,
items[itemIndex].w},
    {items[itemIndex].h, items[itemIndex].w,
items[itemIndex].b},
    {items[itemIndex].h, items[itemIndex].b,
items[itemIndex].w}
};

for (int r = 0; r < 6; r++) {

    int w = rotations[r][0];
    int b = rotations[r][1];
    int h = rotations[r][2];

    for (int x = 0; x <= suitcaseW - w; x++) {
        for (int y = 0; y <= suitcaseB - b;
y++) {
            for (int z = 0; z <= suitcaseH - h;
z++) {

                Placement newBox;

                newBox.x1 = x;
```

```
                newBox.y1 = y;
                newBox.z1 = z;

                newBox.x2 = x + w;
                newBox.y2 = y + b;
                newBox.z2 = z + h;

                if
(!checkCollision(packedCount, newBox)) {


currentPlacement[packedCount] = newBox;

                    packItems(itemIndex + 1,
packedCount + 1);
                }
            }
        }
    }
}

// Skip item
packItems(itemIndex + 1, packedCount);


}

int main() {
```

```
printf("Enter suitcase dimensions (width
breadth height): ");
scanf("%d %d %d", &suitcaseW, &suitcaseB,
&suitcaseH);

printf("Enter number of items: ");
scanf("%d", &totalItems);

for (int i = 0; i < totalItems; i++) {

  printf("Enter dimensions of item %d (width
breadth height): ", i + 1);
    scanf("%d %d %d", &items[i].w, &items[i].b,
&items[i].h);

    items[i].volume = items[i].w * items[i].b *
items[i].h;
}

sortItems();

packItems(0, 0);

printf("\nMaximum items packed: %d\n",
maxPacked);

for (int i = 0; i < maxPacked; i++) {

    printf("\nItem %d\n", i + 1);
```

```
        printf("Bottom-front-left: (%d, %d, %d)\n",
               bestPlacement[i].x1,
               bestPlacement[i].y1,
               bestPlacement[i].z1);

        printf("Top-back-right: (%d, %d, %d)\n",
               bestPlacement[i].x2,
               bestPlacement[i].y2,
               bestPlacement[i].z2);
    }

    return 0;


    }
```

# in devc++:

DAA-capstone.cpp

```cpp
#include <stdio.h>

#define MAX_ITEMS 20

typedef struct {
    int w, b, h;
    int volume;
} Item;

typedef struct {
    int x1, y1, z1;
    int x2, y2, z2;
} Placement;

Item items[MAX_ITEMS];

Placement currentPlacement[MAX_ITEMS];
Placement bestPlacement[MAX_ITEMS];

int suitcaseW, suitcaseB, suitcaseH;
int totalItems;

int maxPacked = 0;

// Sort items by volume (descending)
void sortItems() {

    for (int i = 0; i < totalItems - 1; i++) {
        for (int j = i + 1; j < totalItems; j++) {

            if (items[i].volume < items[j].volume) {

                Item temp = items[i];
                items[i] = items[j];
                items[j] = temp;
            }
        }
    }
}

// Overlap check
int isOverlap(Placement a, Placement b) {

    if (a.x1 >= b.x2 || a.x2 <= b.x1)
        return 0;
```

```
    if (a.z1 >= b.z2 || a.z2 <= b.z1)
        return 0;

    return 1;
}

// Collision check
int checkCollision(int placedCount, Placement newBox) {

    for (int i = 0; i < placedCount; i++) {

        if (isOverlap(currentPlacement[i], newBox))
            return 1;
    }

    return 0;
}

// Backtracking
void packItems(int itemIndex, int packedCount) {

    // Update best result
    if (packedCount > maxPacked) {

        maxPacked = packedCount;

        for (int i = 0; i < packedCount; i++)
            bestPlacement[i] = currentPlacement[i];
    }

    // Stop condition
    if (itemIndex >= totalItems)
        return;

    // PRUNING
    if (packedCount + (totalItems - itemIndex) <= maxPacked)
        return;

    int rotations[6][3] = {
        {items[itemIndex].w, items[itemIndex].b, items[itemIndex].h},
        {items[itemIndex].w, items[itemIndex].h, items[itemIndex].b},
        {items[itemIndex].b, items[itemIndex].w, items[itemIndex].h},
        {items[itemIndex].b, items[itemIndex].h, items[itemIndex].w},
        {items[itemIndex].h, items[itemIndex].w, items[itemIndex].b},
```

DAA-capstone.cpp

```cpp
        {items[itemIndex].b, items[itemIndex].h, items[itemIndex].w},
        {items[itemIndex].h, items[itemIndex].w, items[itemIndex].b},
        {items[itemIndex].h, items[itemIndex].b, items[itemIndex].w}
    };

    for (int r = 0; r < 6; r++) {

        int w = rotations[r][0];
        int b = rotations[r][1];
        int h = rotations[r][2];

        for (int x = 0; x <= suitcaseW - w; x++) {
            for (int y = 0; y <= suitcaseB - b; y++) {
                for (int z = 0; z <= suitcaseH - h; z++) {

                    Placement newBox;

                    newBox.x1 = x;
                    newBox.y1 = y;
                    newBox.z1 = z;

                    newBox.x2 = x + w;
                    newBox.y2 = y + b;
                    newBox.z2 = z + h;

                    if (!checkCollision(packedCount, newBox)) {

                        currentPlacement[packedCount] = newBox;

                        packItems(itemIndex + 1, packedCount + 1);
                    }
                }
            }
        }
    }

    // Skip item
    packItems(itemIndex + 1, packedCount);
}

int main() {

    printf("Enter suitcase dimensions (width breadth height): ");
    scanf("%d %d %d", &suitcaseW, &suitcaseB, &suitcaseH);

```

```c
}

int main() {

    printf("Enter suitcase dimensions (width breadth height): ");
    scanf("%d %d %d", &suitcaseW, &suitcaseB, &suitcaseH);

    printf("Enter number of items: ");
    scanf("%d", &totalItems);

    for (int i = 0; i < totalItems; i++) {

        printf("Enter dimensions of item %d (width breadth height): ", i + 1);
        scanf("%d %d %d", &items[i].w, &items[i].b, &items[i].h);

        items[i].volume = items[i].w * items[i].b * items[i].h;
    }

    sortItems();

    packItems(0, 0);

    printf("\nMaximum items packed: %d\n", maxPacked);

    for (int i = 0; i < maxPacked; i++) {

        printf("\nItem %d\n", i + 1);

        printf("Bottom-front-left: (%d, %d, %d)\n",
                bestPlacement[i].x1,
                bestPlacement[i].y1,
                bestPlacement[i].z1);

        printf("Top-back-right: (%d, %d, %d)\n",
                bestPlacement[i].x2,
                bestPlacement[i].y2,
                bestPlacement[i].z2);
    }

    return 0;
}
```

# Results:

input:

Suitcase: 5 5 5

Items: 3

3 2 1

2 2 2

1 1 1

output:

Maximum items packed: 3

```
C:\Users\nishi\OneDrive\Docu

Enter suitcase dimensions (width breadth height): 5 5 5
Enter number of items: 3
Enter dimensions of item 1 (width breadth height): 3 2 1
Enter dimensions of item 2 (width breadth height): 2 2 2
Enter dimensions of item 3 (width breadth height): 1 1 1

Maximum items packed: 3

Item 1
Bottom-front-left: (0, 0, 0)
Top-back-right: (2, 2, 2)

Item 2
Bottom-front-left: (0, 0, 2)
Top-back-right: (3, 2, 3)

Item 3
Bottom-front-left: (0, 0, 3)
Top-back-right: (1, 1, 4)

--------------------------------
Process exited after 19.36 seconds with return value 0
Press any key to continue . . .
```

## Algorithm Performance:

Time Complexity (Worst Case):

$O((6 \times P)^N)$

Where:

- N = number of items
- 6 = possible rotations per item
- P = number of possible placement positions (W x B x H)

Space Complexity:

O(N)

due to recursion stack and placement storage.

Pruning and volume-based sorting significantly reduce the search space in practical scenarios.

## Github Link:

https://github.com/Nishitha-31/DAA-capstone-project/tree/main

## Conclusion:

This project demonstrates the use of a Backtracking algorithm to solve a three-dimensional packing problem. The algorithm successfully determines the optimal placement of items inside a suitcase while respecting boundary and collision constraints.

The approach highlights important algorithmic concepts such as recursion, search space exploration, pruning, and optimization. While the algorithm may become computationally expensive for very large inputs, it performs effectively for moderate problem sizes.

Future improvements may include the use of heuristic or approximation algorithms to handle larger datasets more efficiently.